DESIGN AND ANALYSIS OF ALGORITHMS

Assignment 3 | Graphs

DSU · MST · SCC · Topological order · DAG shortest paths

10
graph configurations

50
benchmark rows

10/10
reference checks

Method & validation

Every graph is generated with a fresh **Random(42)**. Each counter is updated inside the algorithm. Time is the median of five runs; all conclusions use operation counts. The benchmark compares Kruskal with Prim, validates every DAG order, checks scan identities, and fails before writing a row if a reference value differs.

Reference results

Vertices	Edges	MST weight	SCCs	DAG distance sum
1,000	4,000	147,970	27	396,274
1,000	20,000	30,363	1	252,411
3,162	12,648	468,911	137	1,271,911
3,162	63,240	97,589	1	723,157
10,000	40,000	1,486,139	373	3,941,828
10,000	200,000	309,550	1	2,381,342
31,623	126,492	4,715,515	1,188	12,551,891
31,623	632,460	968,517	1	7,376,612
100,000	400,000	15,012,094	3,923	40,096,539
100,000	2,000,000	3,056,681	1	23,512,087

All values match the assignment reference table. The two MST algorithms return the same total weight on every undirected graph.

VERIFICATION SUITE

JUnit checks DSU against a reference partition, compares 100 random MST pairs, verifies SCCs by brute-force reachability, detects cycles, compares DAG paths with Bellman-Ford, and traverses a path of 100,000 vertices without recursion.

Complexity at a glance

Bounds below exclude the stored input graph. **V** is the vertex count, **E** the edge count, and **α(V)** the inverse Ackermann function.

Algorithm	Time	Extra space	Why
DSU find / union	O(α(V)) amortized	O(V)	Union by size plus path compression.
Kruskal	O(E log E + Eα(V))	O(E + V)	Sort edges; test endpoints with DSU.
Prim (lazy heap)	O((V + E) log(E + V))	O(E + V)	Scan adjacency; push frontier candidates.
Kosaraju	O(V + E)	O(V + E)	Two iterative DFS passes and reverse graph.
Kahn	O(V + E)	O(V)	Indegrees, queue, then one edge traversal.
DAG shortest paths	O(V + E)	O(V)	Topological order and one relaxation pass.

Why the MST choices are safe

Consider any cut separating vertices into two sets. Among edges crossing the cut, choose a minimum-weight edge **e**. If a minimum spanning tree lacks **e**, adding it creates a cycle containing another edge **f** crossing the cut. Replacing **f** with **e** cannot increase the weight, so an MST containing **e** exists. Kruskal applies this to cuts between DSU components and rejects edges that would create cycles. Prim applies it to the cut between visited and unvisited vertices, choosing the lightest frontier edge. Repeating safe choices yields one MST per connected component, including disconnected forests.

Why one DAG pass is enough

A topological order places every predecessor before its successor. Initialize each zero-indegree vertex at distance zero, as if linked from a virtual super-source. Assume distances of all previously processed vertices are final. Every path to the next vertex ends with an edge from an earlier vertex, so its best candidate has already been considered. Relaxing its outgoing edges passes that best distance to its successors. Induction therefore proves that one pass computes every shortest distance, while parent pointers reconstruct a path. Negative edge weights are safe because a DAG cannot revisit a vertex or contain a negative cycle.

Implementation details

ONE GRAPH TYPE

One adjacency-list class supports directed and undirected edges, parallel edges, self-loops, isolated vertices, and disconnected inputs.

STACK SAFETY

Kosaraju uses explicit stacks in both passes. maxDepth records the deepest stack, and the 100,000-vertex path test passes.

CYCLE HANDLING

Kahn returns no order when a cycle remains. DAG shortest paths reject the same graph instead of returning incorrect distances.

Measured work vs formulas

First and last columns compare n = 1,000 and n = 100,000. The ratios stay bounded as the graph grows; exact counts are stored in results.csv.

Algorithm / density	Diagnostic ratio	First	Last
Kruskal / 4n	dsuOps / m	2.290	2.748
Kruskal / 20n	dsuOps / m	0.580	0.894
Kruskal / 4n	comparisons / (m log2 m)	0.891	0.745
Kruskal / 20n	comparisons / (m log2 m)	0.903	0.666
Prim / 4n	comparisons / (m log2 m)	1.829	1.934
Prim / 20n	comparisons / (m log2 m)	1.825	1.890
Prim / both	edgeScans / m	2.000	2.000
Kosaraju / both	edgeScans / m	2.000	2.000
Kahn / both	edgeScans / m	1.000	1.000
DAG paths / both	edgeScans / m	1.000	1.000

Scan behavior in the measured runs

Prim and Kosaraju examine **2m** adjacency entries because each undirected edge appears at both endpoints. Kahn and DAG shortest paths examine **m** directed edges. These identities hold at both tested graph densities.

Why Kruskal scans a variable share

Kruskal sorts the complete edge list, then checks candidate edges with DSU until the spanning forest is complete. Its number of examined edges depends on where the useful edges appear in sorted order. The comparison ratios in the table therefore give a clearer view of sorting cost than its edge-scan count alone.

Which algorithm should I use?

Graph / question	Choose	Reason
Undirected; input is an edge list	Kruskal	Sorting plus DSU handles cycles and forests.
Undirected sparse adjacency list	Prim	Grow directly from adjacent frontier edges.
Undirected dense graph	Measure both	Kruskal sorts all edges; lazy Prim stores many candidates.
Directed; cycles and deadlocks possible	Kosaraju	SCCs group mutually reachable jobs.
Need a legal task order	Kahn	Returns an order or reports a cycle.
Weighted DAG; fastest route	DAG paths	One pass, including negative edge weights.
General directed graph with cycles	Other path method	DAG relaxation is not applicable.

What the slopes say

Least-squares slopes of log(edgeScans) against log(n) are **1.000** for Prim, Kosaraju, Kahn, and DAG relaxation at both densities. Kruskal gives **1.055** (4n) and **1.119** (20n) because its examined fraction grows. That scan slope does not prove the sorting logarithm: comparison counts divided by m log2 m are the relevant diagnostic.

Timing is context, not the main evidence

At n = 100,000, median Kruskal times were **168.38 ms** (4n) and **977.79 ms** (20n); Prim took **290.58 ms** and **1912.51 ms**. Hardware, JVM warmup, garbage collection, and background load change these figures. The reference values and operation counters are the reproducible evidence.